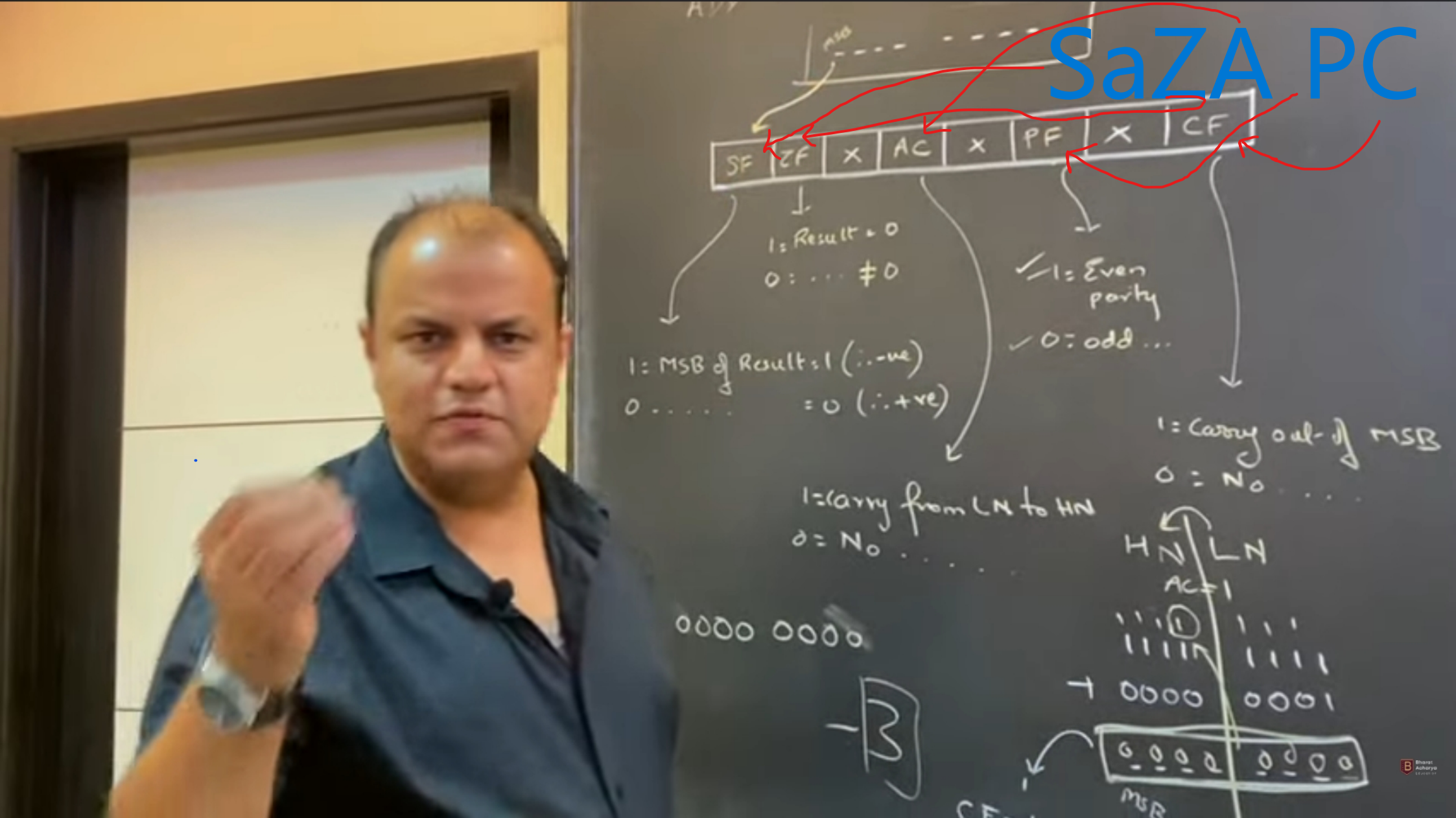
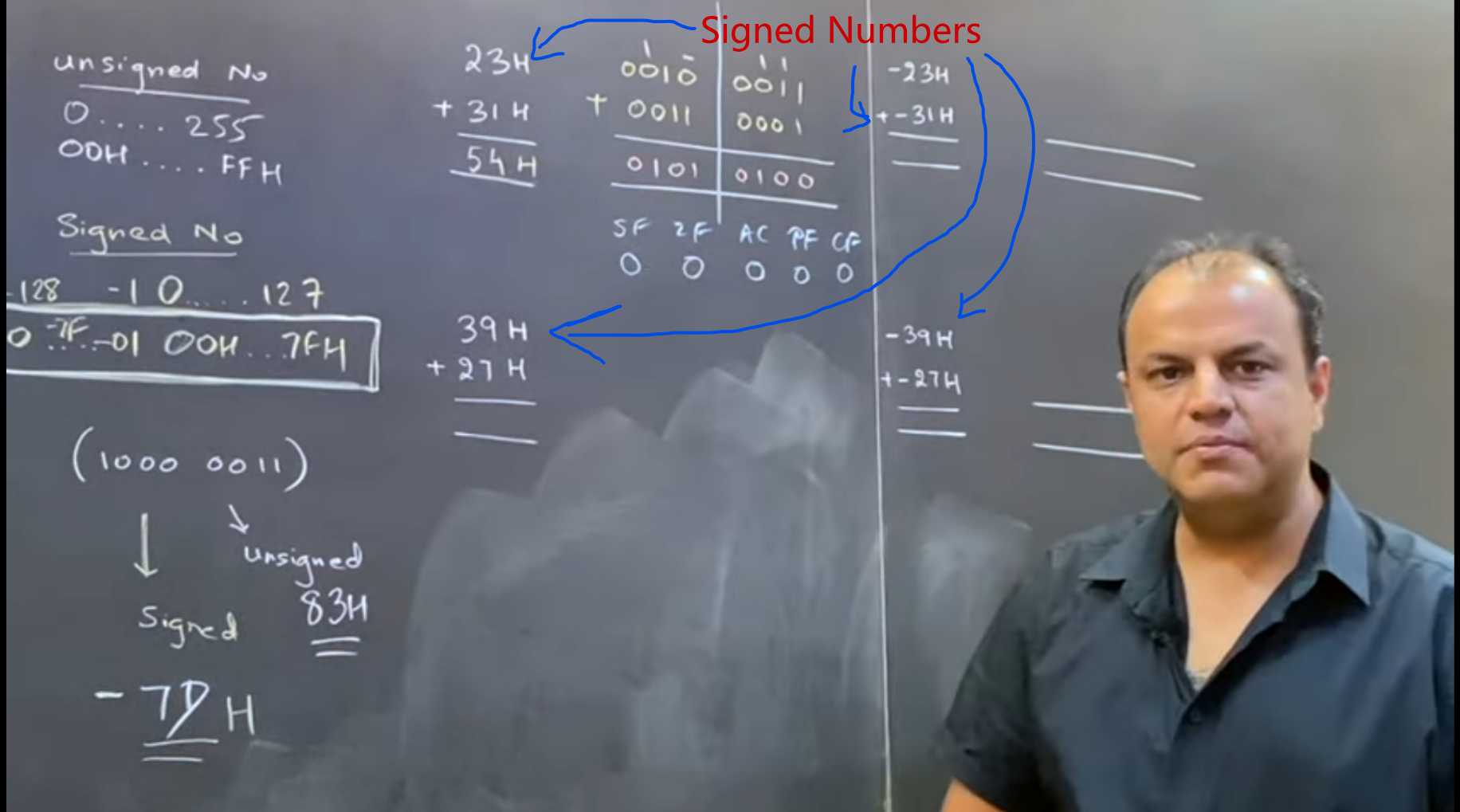
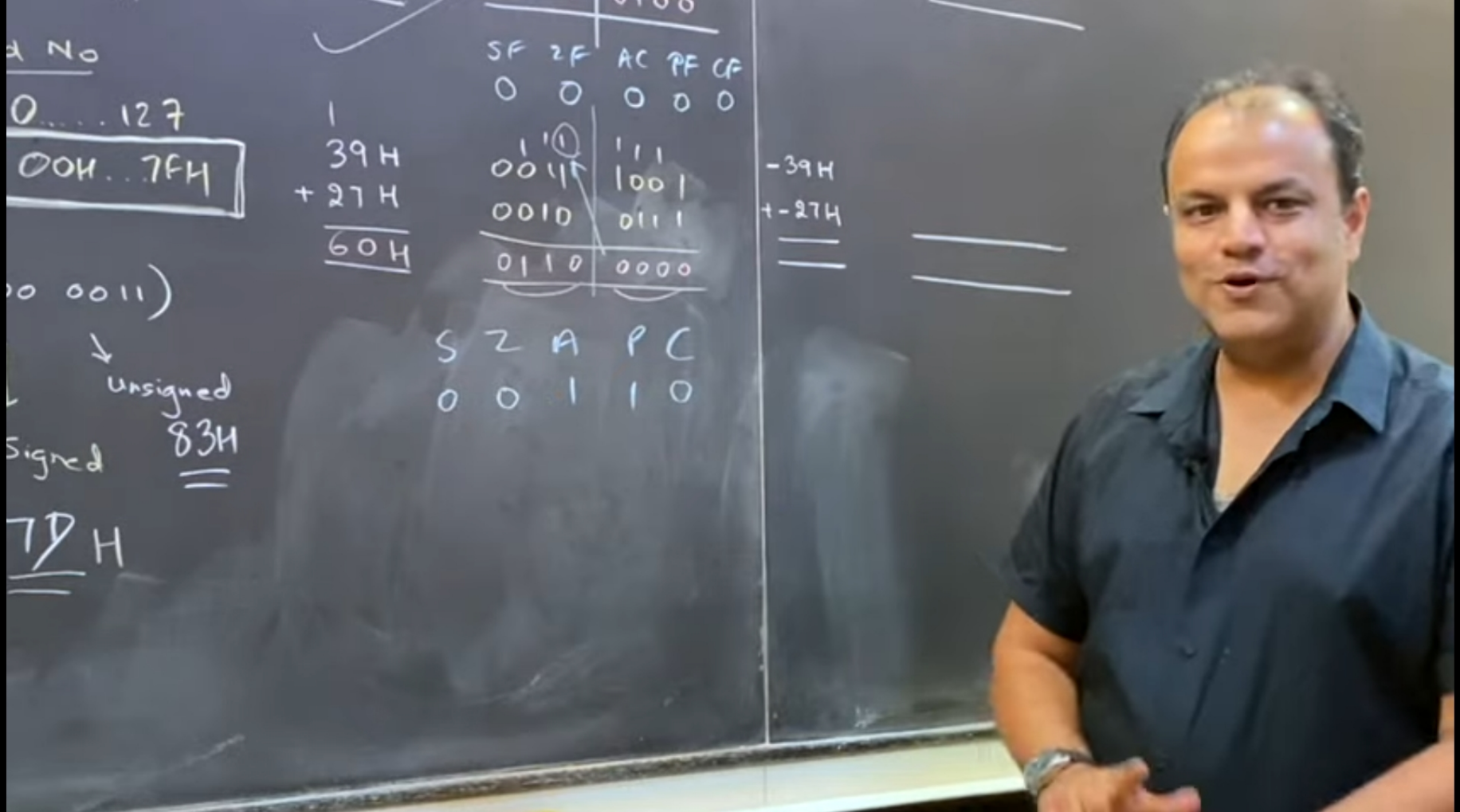
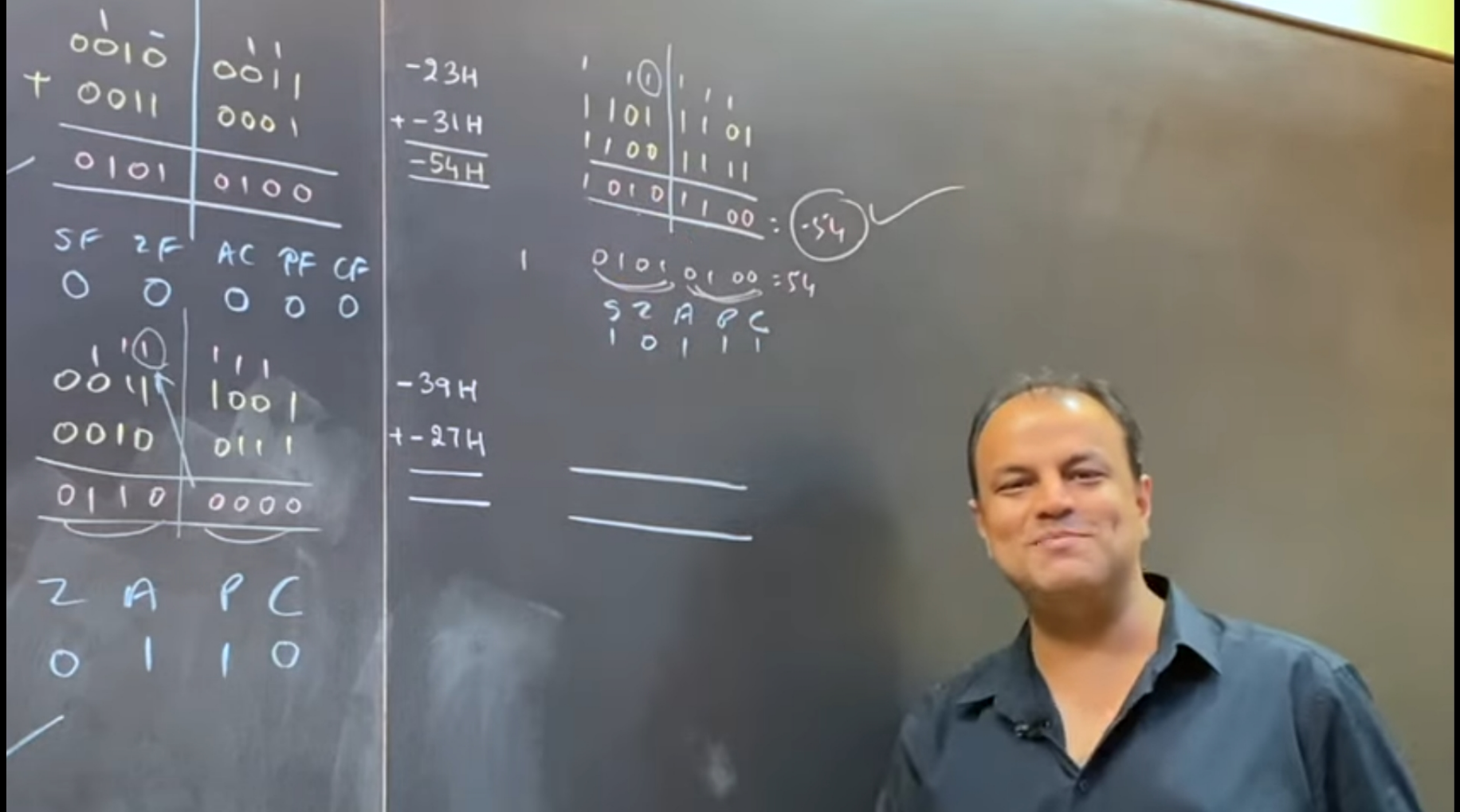
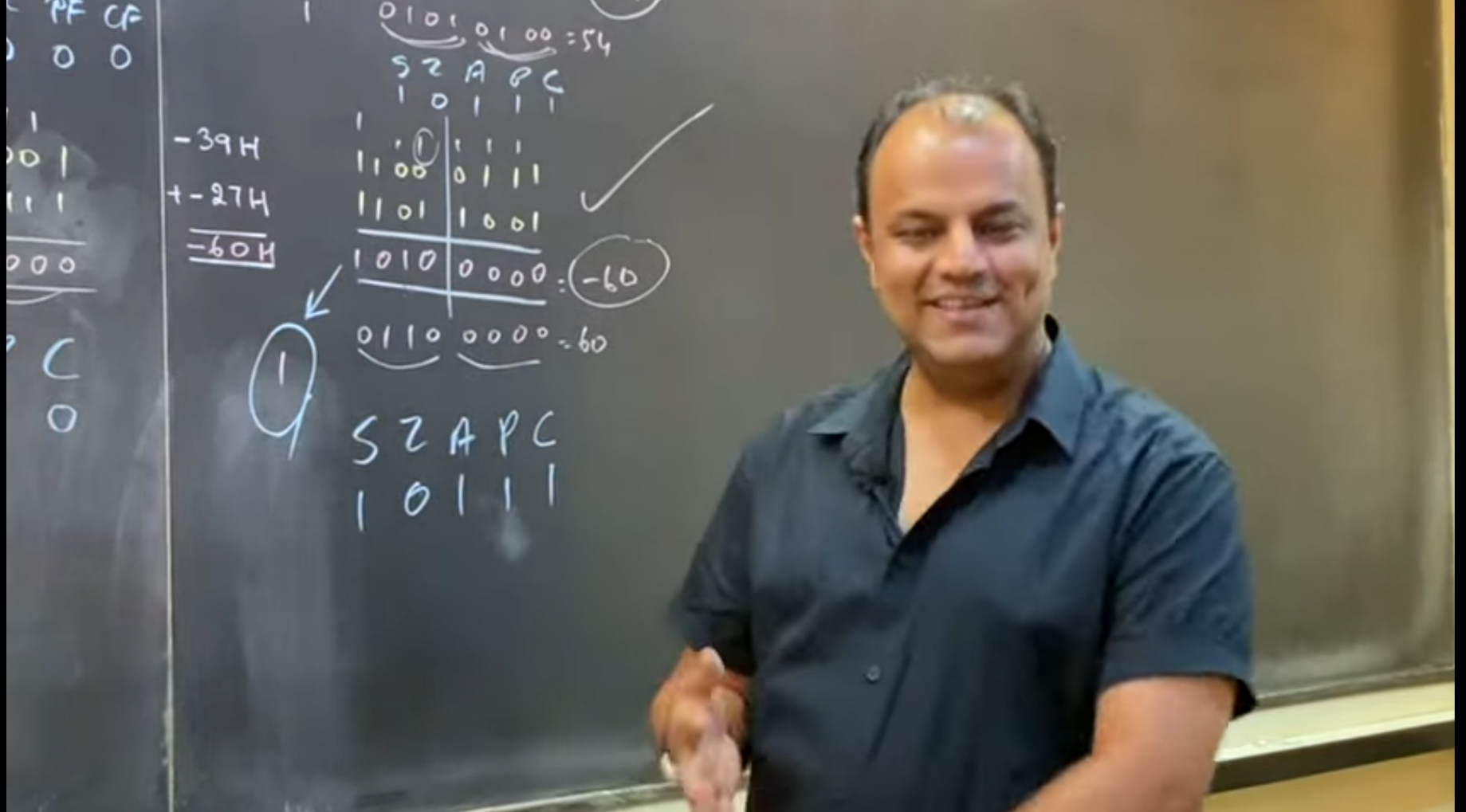

SaZA PC
SF ZF X AC X PF X CF
1: Result = 0
0: ... ≠0
1: Even parity
0: odd ...
1: MSB of Result = 1 (:-ve)
0 ..... = 0 (:+ve)
1: carry from LN to HN
0: No
1: Carry out of MSB
0: No
HN LN
AC=1
0000 0001
0000 0000
-3
MSB


Signed Numbers
unsigned No
0 255
00H .... FFH
Signed No
-128 -1 0 127
80 .. FF 01 00H .. 7FH
(1000 0011)
Signed
Unsigned
83H
-7DH
23H
+31H
54H
0010 0011
0011 0001
0101 0100
SF ZF AC PF CF
0 0 0 0 0
-23H
+-31H
39H
+27H
-39H
+-27H


d No
0 . . . 127
00H . . 7FH
00 0011)
Unsigned
Signed
83H
7D H
1
39 H
+ 27 H
60 H
SF ZF AC PF CF
0 0 0 0 0
1 1 1
0011 1001
0010 0111
0110 0000
S Z A P C
0 0 1 1 0
-39H
+-27H


0010
0011
0101
0011
0001
0100
SF ZF AC PF CF
0 0 0 0 0
-23H
+-31H
-54H
1101
1100
0101
1101
1111
1100
: -54
0101 0100 = 54
S Z A P C
1 0 1 1 1
0011
0010
0110
1111
1001
0111
0000
-39H
+-27H
Z A P C
0 1 1 0


-39H
+-27H
-60H
S Z A P C
1 0 1 1 1
S Z A P C
1 0 1 1 1

Flags register in 8085 Microprocessor

In **8085 microprocessor**, the flags register can have a total of eight flags. Thus a flag can be represented by 1 bit of information. But **only five flags are implemented** in 8085. And they are:

- Carry flag (Cy),
- Auxiliary carry flag (AC),
- Sign flag (S),
- Parity flag (P), and
- Zero flag (Z).

The **"x" - is don't care bits** in the flags register. The user is not required to memorize the positions of these flags in the flags register.

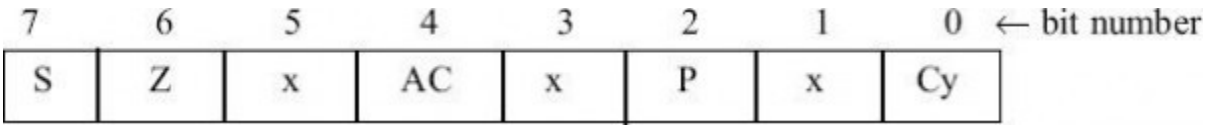


Fig. Flags register

Now consider the programmer's view of 8085 contains the flags register has been depicted in the following figure -

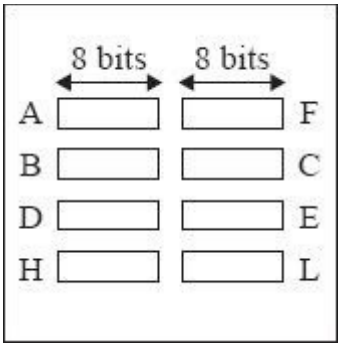


Fig. Programmer's view of 8085 Flags register

These individual flags are either set to 1, or 0 depending on the result of execution of the last executed arithmetic or logical instruction. But in a few arithmetic and logical instructions, some or none of these flags are affected. E.g., in the execution of DCX and INX instructions, flag register will not get affected at all.

Also, in any data transfer instruction, none of the flags bits in flag register are affected.

Carry flag (Cy): after performing the **addition of any two 8-bit numbers**, the **carry** generated can be **either 0 or 1**. The Cy flag is stored in the LS bit position in the flags register.

Example 1: In the addition of 45H and F3H, the result thus produced will be 38H and with Cy flag = 1, as shown below.

```
Cy AC
 10
   4 5H
 +F 3H
-------------
  3 8H = 0011 1000
```

Example 2: In the addition of 85H and 1EH, the result thus produced will be A3H with Cy = 0, as shown below.

```
Cy AC
 01
   8 5H
 +1 EH
-------------
  A 3H = 1010 0011
```

Auxiliary carry flag (Ac): Consider the addition of any two 8-bit (2-hex digit) numbers, in which a **carry** may be **generated** when we **add** the **LS hex digits** of the two numbers. Such a carry is called intermediate carry also known as half carry, or **auxiliary carry (AC).** Intel prefers to call it AC. In the above Example 1, AC was not generated but in Example 2, AC is generated.

Sign flag (S): The S flag is set to **1**, when the **result** thus produced against any logical or arithmetic operations is **negative, indicated by MS bit of 8-bit result being 1**. It is reset to **0** otherwise if the **result** is **positive, indicated by MS bit of 8-bit result being 0**.

When we shall work with **unsigned numbers**, We shall simply **ignore the S flag**. For example, if we are treating 85H and 1EH as unsigned numbers, their sum will be the unsigned number A3H. In this case, S flag becomes 1, but we do not care for the value of the S flag. And we shall ignore it as well.

Parity flag (P): The **P flag is set to 1**, if the 8-bit result thus produced against any logical and arithmetic operation has an **even number of 1's** in it. If there are **odd number of 1's** in the 8-bit result, the **P flag is reset to 0.**

As the user does not really care for the number of 1's present in the result after an arithmetic operation, this flag is not of much use practically.

Zero flag (Z): The Z flag is set to **1**, if after arithmetic and logical operations, the 8-bit **result** thus **produced, is 00H**. If the 8-bit **result is not equal to 00H**, the **Z flag is reset to 0.** Thus the Z flag is hoisted to indicate that the result is 0